

WSN Energy-Harvesting Routing Simulator

Full-Stack Web Application Execution & Setup Guide (FastAPI Backend + React Frontend)

GitHub Repository: https://github.com/santhoshvellore7119-web/WSN_proj | Branch: master | Author: Santhosh

1. Full-Stack System Architecture Overview

The project consists of a complete full-stack web application integrating an asynchronous Python FastAPI backend with an interactive React 19 single-page dashboard. The frontend visualizes node topology, live multi-hop routing paths, and round-by-round energy harvesting telemetry in real time.

Layer	Tech Stack & Port	Role & Responsibilities
Frontend UI	React 19, TypeScript, Vite, Tailwind CSS, Recharts Port 3000	Interactive single-page application. Features real-time SVG sensor topology canvas, round-by-round replay controls, and dynamic residual energy line charts.
Backend API	FastAPI, Uvicorn, SQLite, SQLAlchemy, Pydantic Port 8000	Asynchronous REST API. Manages background simulation jobs, SQLite database persistence (backend/wsn_simulator.db), and provides OpenAPI / Swagger documentation.
Simulation Engine	Python 3.10+, NumPy, SciPy (Core Library in src/)	Executes 3D Time-Augmented DP (\$dp[v][h][t]\$), DSU live detour recovery, LEACH clustering, 1st-order radio model, and solar harvesting profiles.

2. Prerequisites & One-Time Initial Setup

Ensure the following tools are installed on your machine before running: **Python 3.10+** (with pip), **Node.js 18+** (with npm), and **Git**.

Step 1: Clone the Repository

```
git clone https://github.com/santhoshvellore7119-web/WSN_proj.git
cd WSN_proj
```

Step 2: Setup Python Virtual Environment & Install Backend Dependencies

- Windows (CMD / PowerShell):
python -m venv .venv
.venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements.txt
- Linux / macOS:
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

Step 3: Install Frontend Node Dependencies

```
cd frontend
npm install
cd ..
```

3. How to Run the Full-Stack Application in Command Prompt / Terminal

To run both the backend API and frontend UI concurrently, open **two separate terminal windows** from the project root:

Terminal 1: Start the FastAPI Backend (Port 8000)

- Activate virtual environment and launch Uvicorn with auto-reload:

```
# Windows CMD / PowerShell:
.venv\Scripts\activate
python -m uvicorn backend.main:app --reload --port 8000

# Or navigate into backend folder:
cd backend
python -m uvicorn main:app --reload --port 8000
```

- Backend will be live at: **http://localhost:8000**
- Interactive API Documentation (Swagger UI): **http://localhost:8000/docs**

Terminal 2: Start the React Frontend (Port 3000)

- Navigate into the frontend folder and start the dev server:

```
cd frontend
npm run dev
```

- Frontend Web Dashboard will be live at: **http://localhost:3000**
- Automatic API Proxying: frontend/server.ts automatically forwards all /api/* requests to http://127.0.0.1:8000.

4. Step-by-Step Procedure to Run in Visual Studio Code (VS Code)

Follow these steps for a clean, integrated development experience inside VS Code:

- Step 1: Open Project in VS Code**
Launch VS Code in the project root: `code .`
- Step 2: Select Python Interpreter**
 - Press `Ctrl + Shift + P` (or `Cmd + Shift + P` on macOS).
 - Type and select **Python: Select Interpreter**.
 - Choose the workspace virtual environment interpreter: `\\.venv\\Scripts\\python.exe` (or `\\.venv\\bin\\python`).
- Step 3: Open Integrated Split Terminals**
 - Open the integrated terminal with `Ctrl + `` (backtick).
 - Click the **Split Terminal** button on the top right of the terminal panel (or press `Ctrl + \`).
 - In the **Left Pane (Backend)**, run:
`cd backend && python -m uvicorn main:app --reload --port 8000`
 - In the **Right Pane (Frontend)**, run:
`cd frontend && npm run dev`

Step 4: Optional 1-Click Debugging via .vscode/launch.json
You can configure VS Code debugger to launch FastAPI with breakpoint support by adding to `.vscode/launch.json`:

```
{ "version": "0.2.0", "configurations": [ { "name": "Debug FastAPI Backend", "type": "debugpy", "request": "launch", "module": "uvicorn", "args": ["backend.main:app", "--reload", "--port", "8000"], "jinja": true } ] }
```

5. Running with Docker Compose (1-Command Full-Stack Boot)

If you have Docker Desktop installed, you can boot both the FastAPI backend and React frontend simultaneously without configuring local environments:

```
# Build and start all containers in detached mode:
docker compose up --build -d

# View live logs from both services:
docker compose logs -f

# Stop and clean up containers:
docker compose down
```

• **Web Dashboard:** <http://localhost:3000> | • **FastAPI Swagger Docs:** <http://localhost:8000/docs>

6. Key REST API Endpoints Overview

Method & Route	Request Body / Params	Functionality
POST /api/run-simulation	JSON: { num_nodes, rounds, harvesting_profile, enable_time_dp, ... }	Triggers asynchronous simulation in background task. Returns job ID.
GET /api/simulation/{id}	Path param: job_id (UUID)	Polls live execution status, node topology coordinates, and round telemetry.
GET /api/scenarios	None	Returns pre-configured scenario templates (Baseline, Solar, Shadowed, Stochastic).
GET /api/history	Query: limit, offset	Retrieves paginated past simulation runs stored in SQLite.
GET /api/export/{id}/csv	Path param: job_id	Exports round-by-round energy, alive nodes, and packet metrics as CSV.

7. Troubleshooting & Common Fixes

Symptom / Error	Cause	Fix
PowerShell script execution disabled	Windows policy blocks activate.ps1.	Run in PowerShell: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
Port 8000 or 3000 already in use	Previous server process is still running.	Windows: netstat -ano findstr :8000 then taskkill /PID <PID> /F
502 Bad Gateway / Backend unavailable	Frontend proxy cannot connect to port 8000.	Ensure FastAPI backend is running in Terminal 1 before opening the frontend in Terminal 2.

8. Quick Full-Stack Execution Cheat-Sheet

```
# 1. Clone & Setup
git clone https://github.com/santhoshvellore7119-web/WSN_proj.git && cd WSN_proj
python -m venv .venv && .venv\Scripts\activate && pip install -r requirements.txt
cd frontend && npm install && cd ..

# 2. Start Backend (Terminal 1)
python -m uvicorn backend.main:app --reload --port 8000

# 3. Start Frontend (Terminal 2)
cd frontend && npm run dev

# 4. Open in Browser
http://localhost:3000 (React App) | http://localhost:8000/docs (Swagger API)
```